



# Transformer

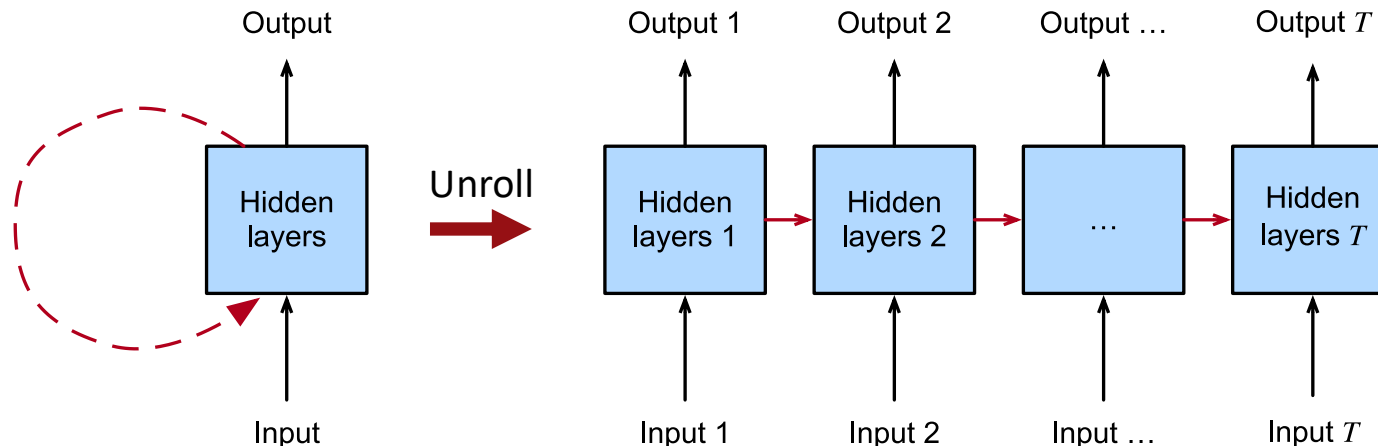
Qian Wang

School of Biomedical Engineering  
ShanghaiTech University

# Recurrent neural network (RNN)

---

- Many learning tasks require dealing with **sequential data**.
  - Tasks such as translating passages of text from one natural language to another, engaging in dialogue, or controlling a robot, demand that models both ingest and output sequentially structured data.
- **RNN**: captures the dynamics of sequences via recurrent connections, which can be thought of as **cycles** in the network of nodes.



# Hidden State

---

- The **hidden state** in RNN serves to capture information about the previous inputs and helps in making the current prediction.
  - $h_t$ : stores the sequence information learned **from the start to the current time  $t$** .
  - At each time time step, RNN takes an input and updates its hidden state.

$$h_t = f(x_t, h_{t-1}).$$

- Don't forget the first-order **Markov** property.

$$P(x_t \mid x_{t-1}, \dots, x_1) \approx P(x_t \mid h_{t-1}),$$

- The hidden state not only affects subsequent hidden states but also influences the output.

# RNN: Math Formulation

- Assuming simple MLP (with single hidden layer, but not hidden state), we have:

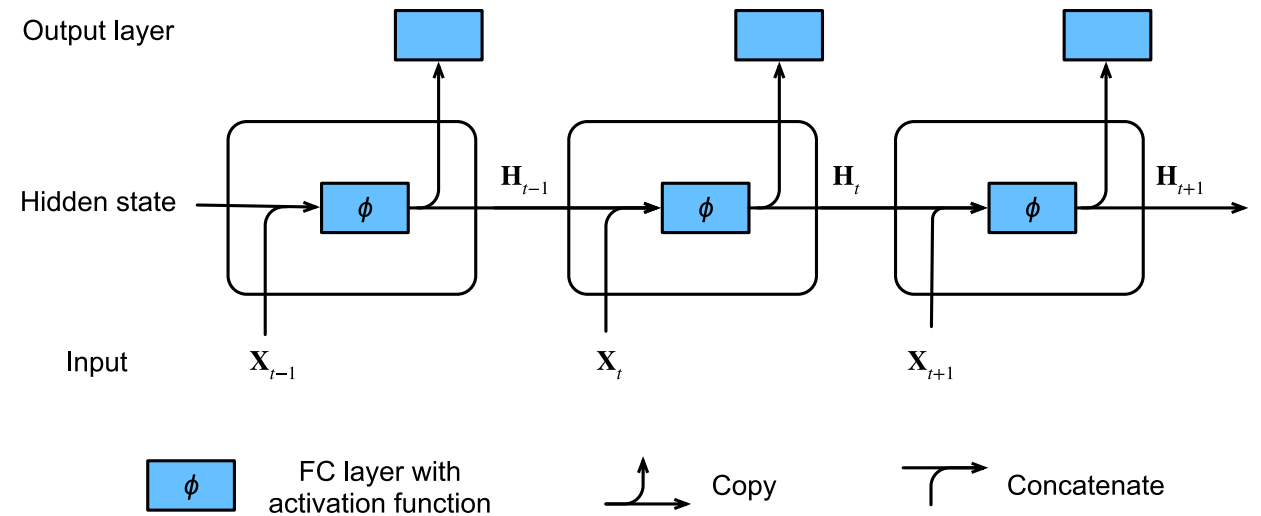
$$\mathbf{H} = \phi(\mathbf{X}\mathbf{W}_{\text{xh}} + \mathbf{b}_{\text{h}}).$$

$$\mathbf{O} = \mathbf{H}\mathbf{W}_{\text{hq}} + \mathbf{b}_{\text{q}},$$

- Based on MLP, accomadate hidden state that is temporally dynamic:

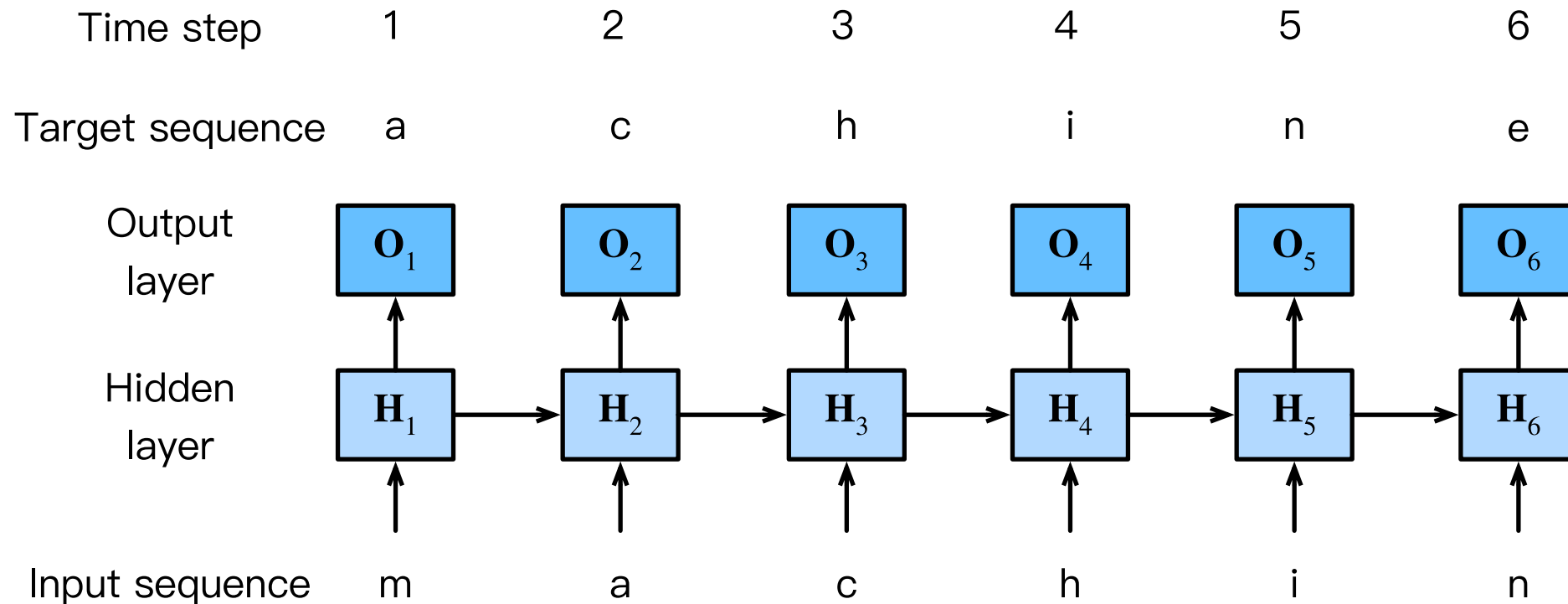
$$\mathbf{H}_t = \phi(\mathbf{X}_t\mathbf{W}_{\text{xh}} + \mathbf{H}_{t-1}\mathbf{W}_{\text{hh}} + \mathbf{b}_{\text{h}}).$$

$$\mathbf{O}_t = \mathbf{H}_t\mathbf{W}_{\text{hq}} + \mathbf{b}_{\text{q}}.$$



# Example: Character-level Language Model

---



Example of **Autoregression**: One can provide an initial input character of “m”, and RNN will output the rest characters of “achine”.

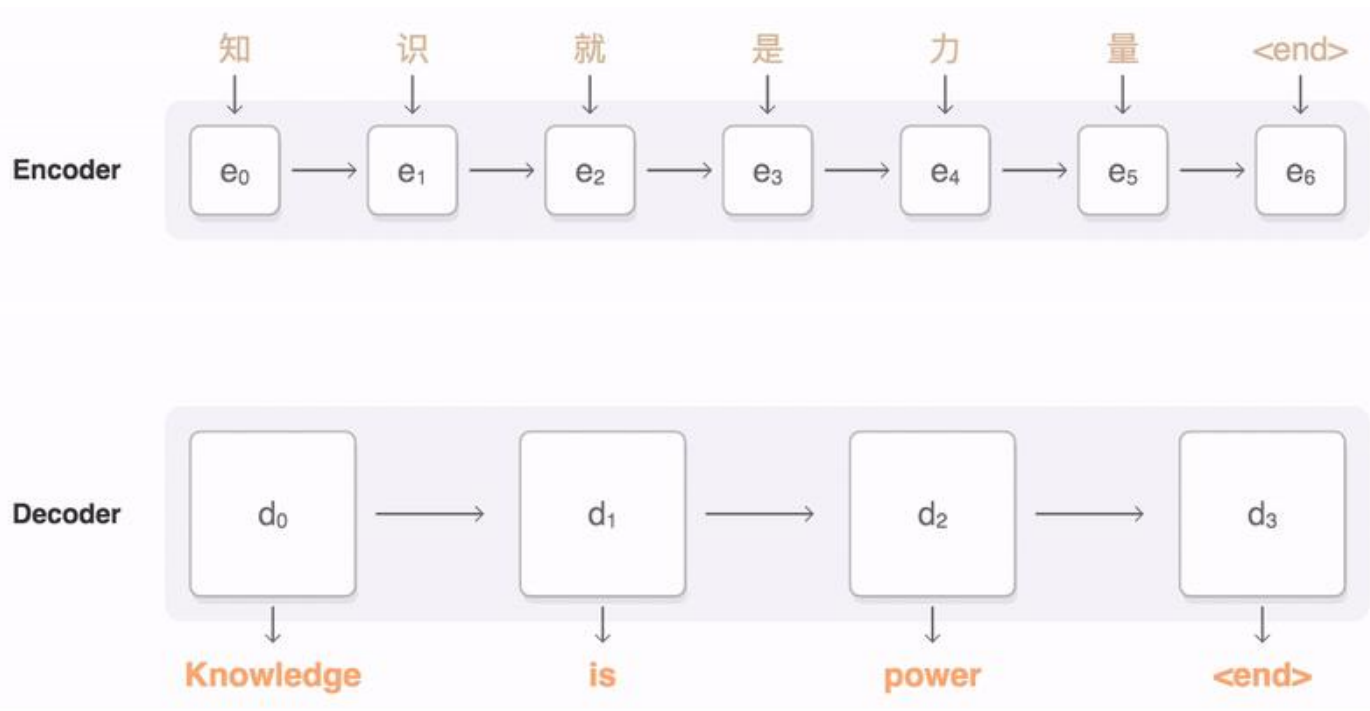
# Backpropagation Through Time (BPTT)

---

- **Training of RNN:** BPTT adapts backpropagation to the temporal nature of RNN by unrolling the network through time and computing gradients for each time step.
  1. **Unrolling:** The RNN is unrolled across the time steps of the input sequence;
  2. **Forward Pass:** The input sequence is processed one element at a time, updating the hidden state and generating outputs;
  3. **Loss Calculation:** The loss between the predicted outputs and the actual target values is measured at each time step;
  4. **Backward Pass:** This involves using the chain rule to propagate errors backward through the unrolled network, hence “through time”;
  5. **Gradient Accumulation:** The gradients at each time step are accumulated, taking into account the contributions from all time steps where a parameter was involved;
  6. **Weight Update:** After accumulating the gradients, the weights of the RNN are updated using optimization algorithms.

# Sequence-to-Sequence (seq2seq)

- Sequence-to-sequence: transform one sequence into another.
  - Typical problems include language translation, summarization, and speech recognition.



---

## Sequence to Sequence Learning with Neural Networks

---

Ilya Sutskever  
Google  
ilyasu@google.com

Oriol Vinyals  
Google  
vinyals@google.com

Quoc V. Le  
Google  
qvl@google.com

# Sequence-to-Sequence (seq2seq)

---

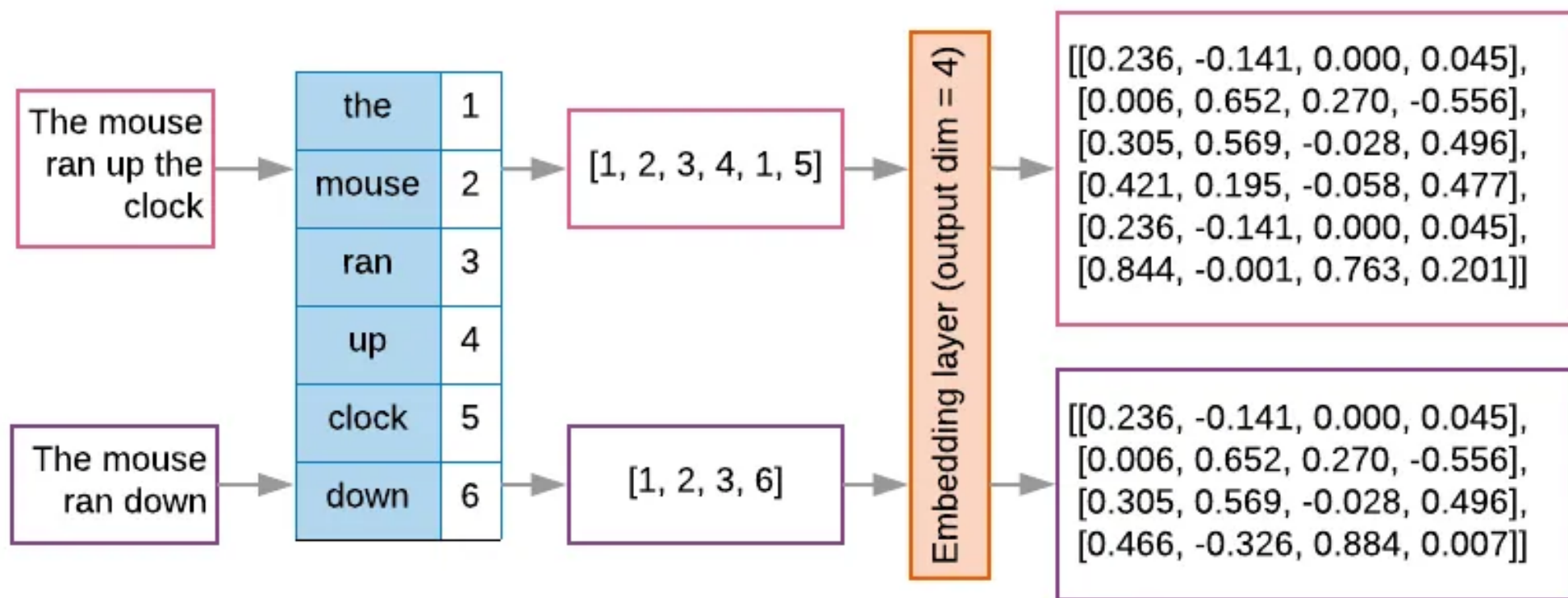
- As inputs and outputs each consist of variable-length unaligned sequences, we generally rely on **encoder-decoder architectures**.
- The **encoder RNN** will take a variable-length sequence as input and transform it into a **fixed-shape hidden state**.
- To generate the output sequence, one token at a time, the **decoder RNN** will predict each successive target token **given both the input sequence and the preceding tokens in the output**.

# Token

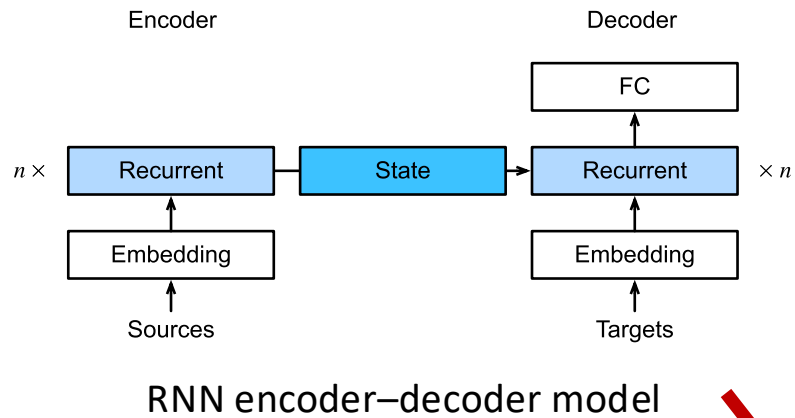
---

- “**Token**” is an elementary piece of information that can be words, subwords, characters, or even bytes.
  - In natural language processing (NLP), a sentence or document is converted into a sequence of tokens before being fed into a model for tasks such as translation.
  - The process of converting text into tokens is called **tokenization**, which can be done at different levels (word-level, subword-level, character-level).
  - Once text is tokenized, each token is typically mapped to a feature representation known as an **embedding**.

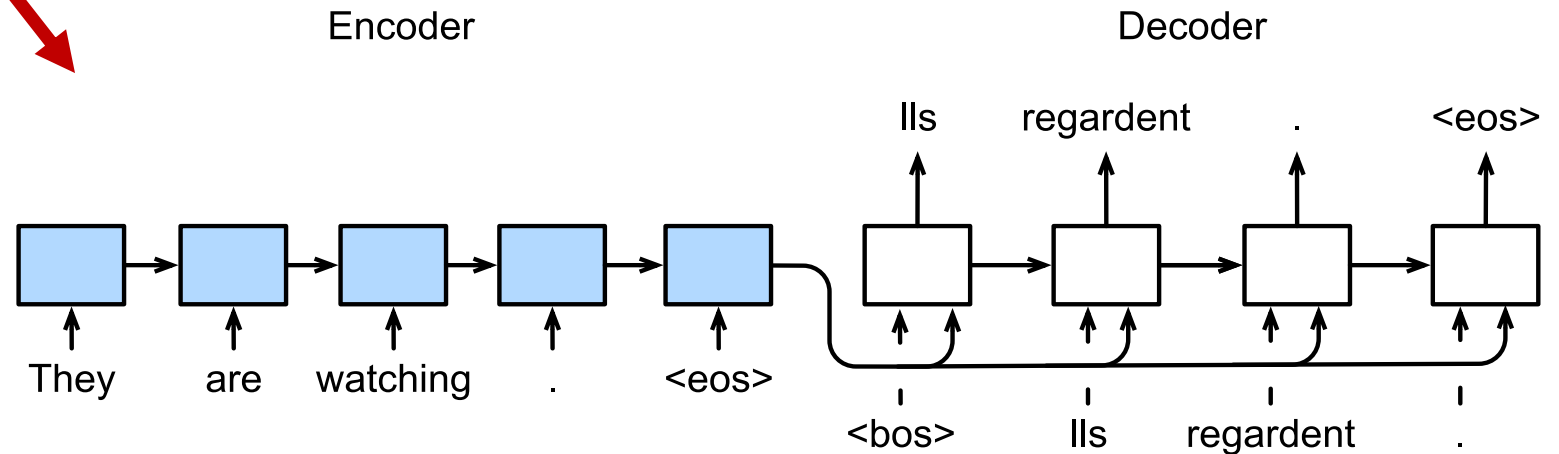
# Token



# Sequence-to-Sequence (seq2seq)

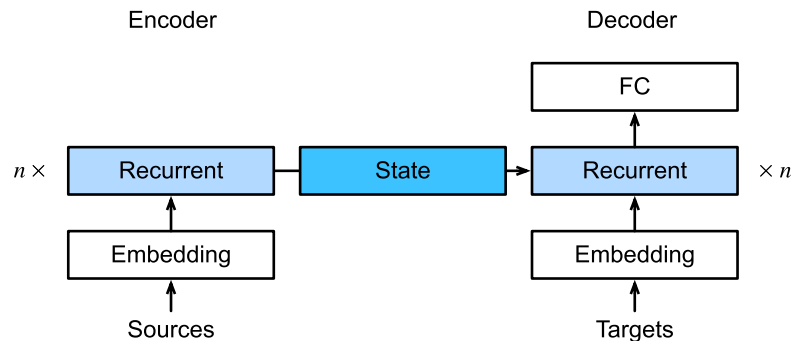


- Seq2seq maps an input sequence into a real-numerical vector by a neural network (the encoder), then map it back to an output sequence using another neural network (the decoder).



Predicting the output sequence token by token using an RNN encoder-decoder

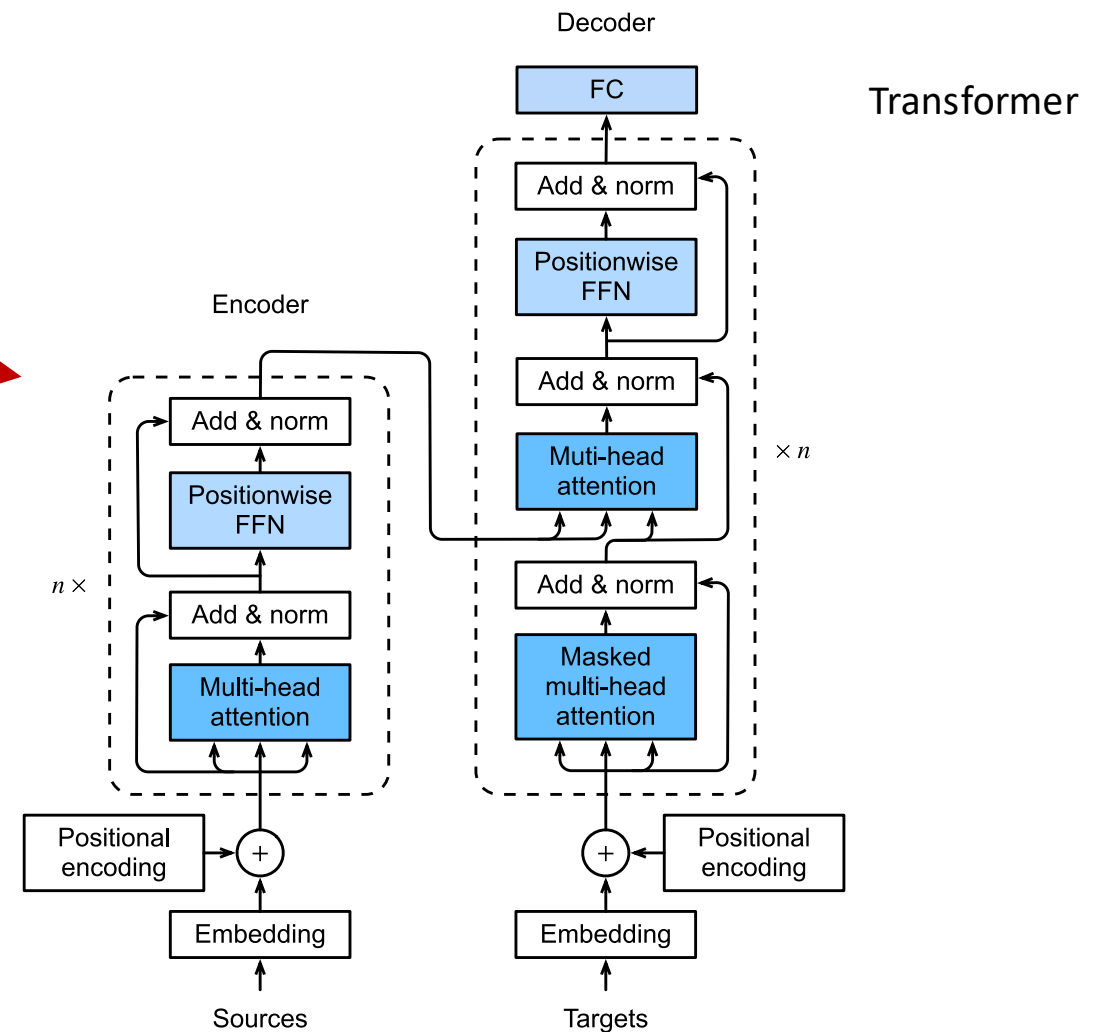
# Transformer



RNN encoder-decoder model

## Limitations of RNN

- Process input data sequentially and thus slowly;
- Struggle with learning long-range dependencies;
- Maintain a hidden state that must encapsulate all relevant information about the sequence seen so far.



---

# Attention Is All You Need

---

**Ashish Vaswani\***  
Google Brain  
avaswani@google.com

**Noam Shazeer\***  
Google Brain  
noam@google.com

**Niki Parmar\***  
Google Research  
nikip@google.com

**Jakob Uszkoreit\***  
Google Research  
usz@google.com

**Llion Jones\***  
Google Research  
llion@google.com

**Aidan N. Gomez\* †**  
University of Toronto  
aidan@cs.toronto.edu

**Łukasz Kaiser\***  
Google Brain  
lukaszkaizer@google.com

**Illia Polosukhin\* ‡**  
illia.polosukhin@gmail.com

# Transformer: Input

---

- The input to a Transformer is typically a sequence of tokens.
  - In NLP tasks, these tokens often represent words, subwords, or characters extracted from the input text.
- Each token is associated with a unique embedding that represents it in a continuous vector space.
- The encoder processes the entire sequence simultaneously (using the self-attention mechanism), to produce a set of encoded representations that capture contextual information about each token in relation to the entire input sequence.

# Transformer: Output

---

- The output is also a sequence of tokens.
  - In tasks like machine translation, this could be a sequence of tokens in the target language.
  - For text summarization, it's a summary of the input text.
- The decoder generates each token one at a time, using both the encoded input sequence from the encoder and the sequence of tokens already generated in the current output sequence to predict the next token.
  - Autoregression: At inference time, the decoder starts with a special start token and generates the output sequence one token at a time, each time updating the sequence with the newly predicted token.

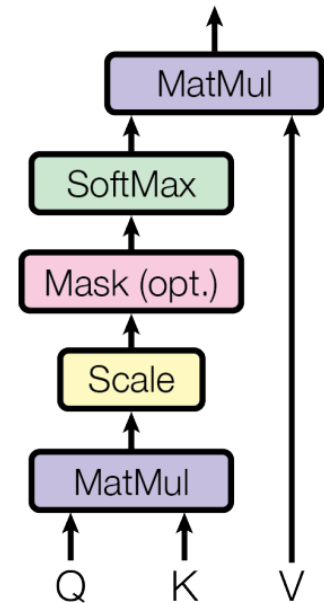
# Query-Key-Value (Q-K-V)

---

- **Query (Q)**: a vector representation that seeks to find relevant information or context from the input data.
  - It acts as a probe or request for information.
- **Key (K)**: a vector representation that provides a label or identifier for a particular piece of information within the input sequence.
  - Keys are used to match Queries.
- **Value (V)**: the actual content or data that we want to be retrieved or attended to, based on the match between Queries and Keys.
  - Values hold the information that will be used in subsequent layers or steps.

# Scaled Dot-Product Attention

- In summary, Q-K-V are abstractions that are useful for calculating **attention**.
  - For a given Q, compute a score with each K in the input.
  - Convert these scores into weights using a softmax function. This normalizes the scores into a probability distribution, indicating the contribution of each input token.
  - Use these weights to compute a weighted sum of V, resulting in an output that emphasizes information from the input sequence according to the learned attention.



$$\begin{aligned}\alpha(\mathbf{q}, \mathbf{k}_i) &= \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) \\ &= \frac{\exp(\mathbf{q}^\top \mathbf{k}_i / \sqrt{d})}{\sum_{j=1} \exp(\mathbf{q}^\top \mathbf{k}_j / \sqrt{d})}.\end{aligned}$$

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 (  $\sqrt{d_k}$  )

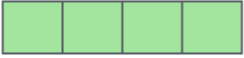
Softmax

Softmax

X  
Value

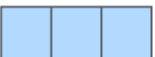
Sum

Thinking

$x_1$  

$q_1$  

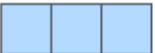
$k_1$  

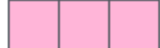
$v_1$  

$q_1 \cdot k_1 = 112$

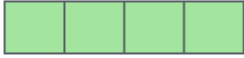
14

0.88

$v_1$  

$z_1$  

Machines

$x_2$  

$q_2$  

$k_2$  

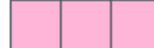
$v_2$  

$q_1 \cdot k_2 = 96$

12

0.12

$v_2$  

$z_2$  

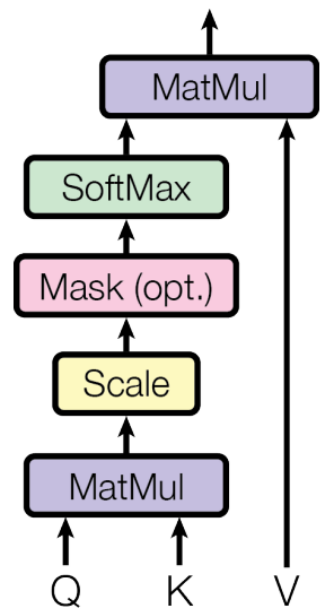
1. Create vectors for Q, K and V ([see next page](#));

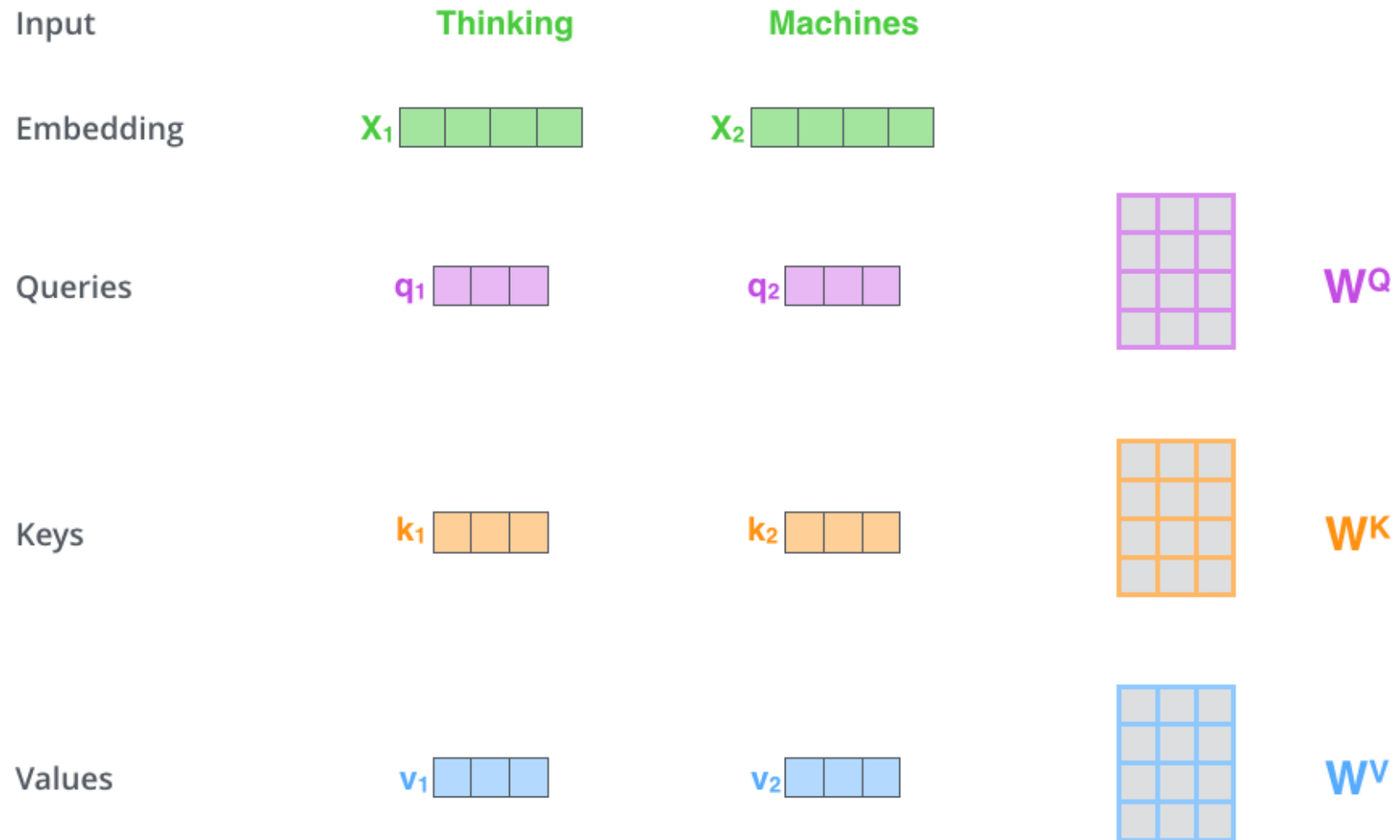
2. Calculate attention scores for each Q;

3. Normalize;

4. Multiple each V by the attention score;

5. Sum up the weighted V vector.

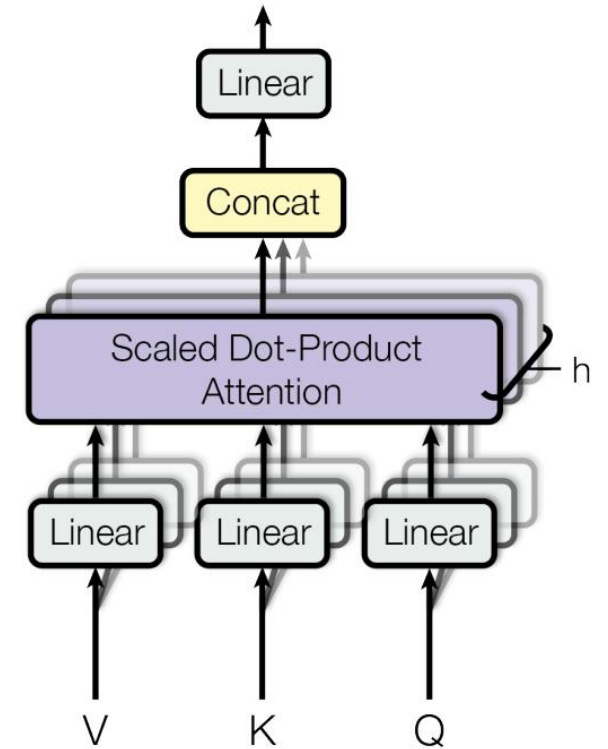


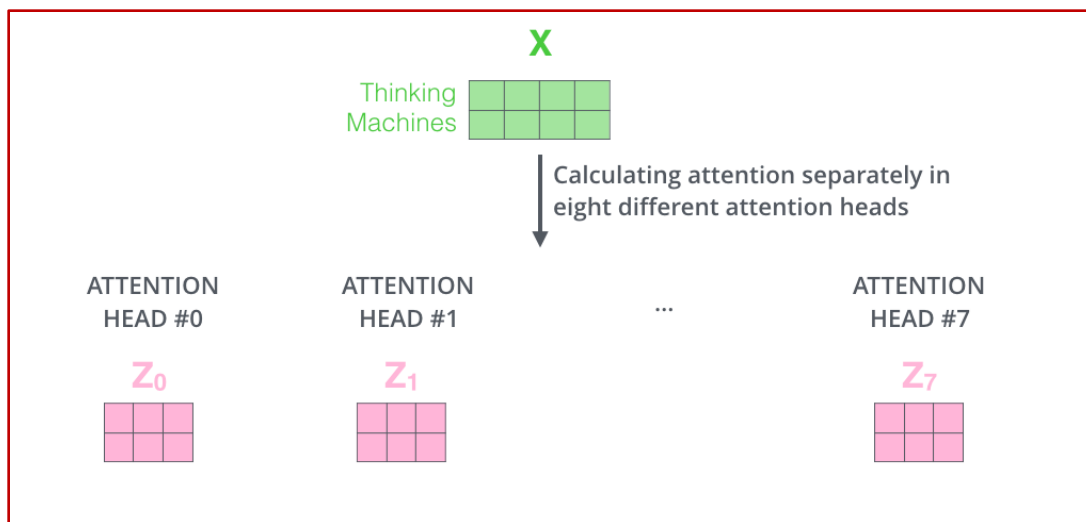
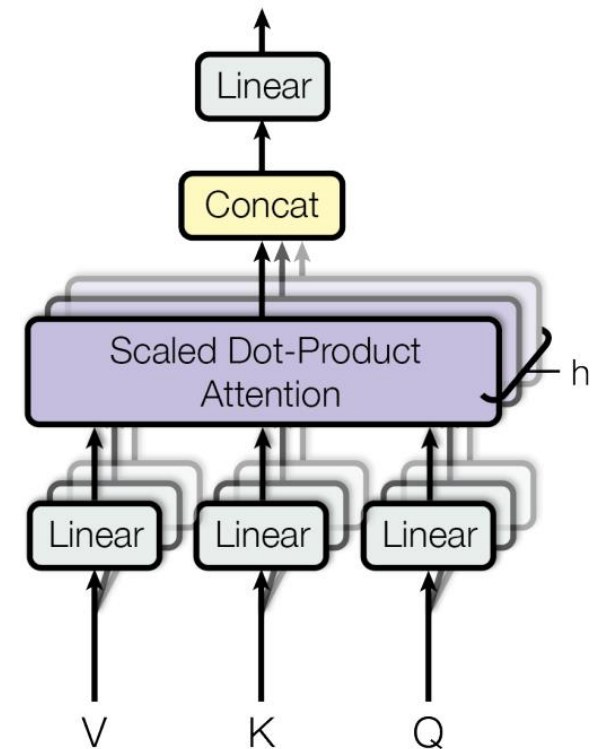
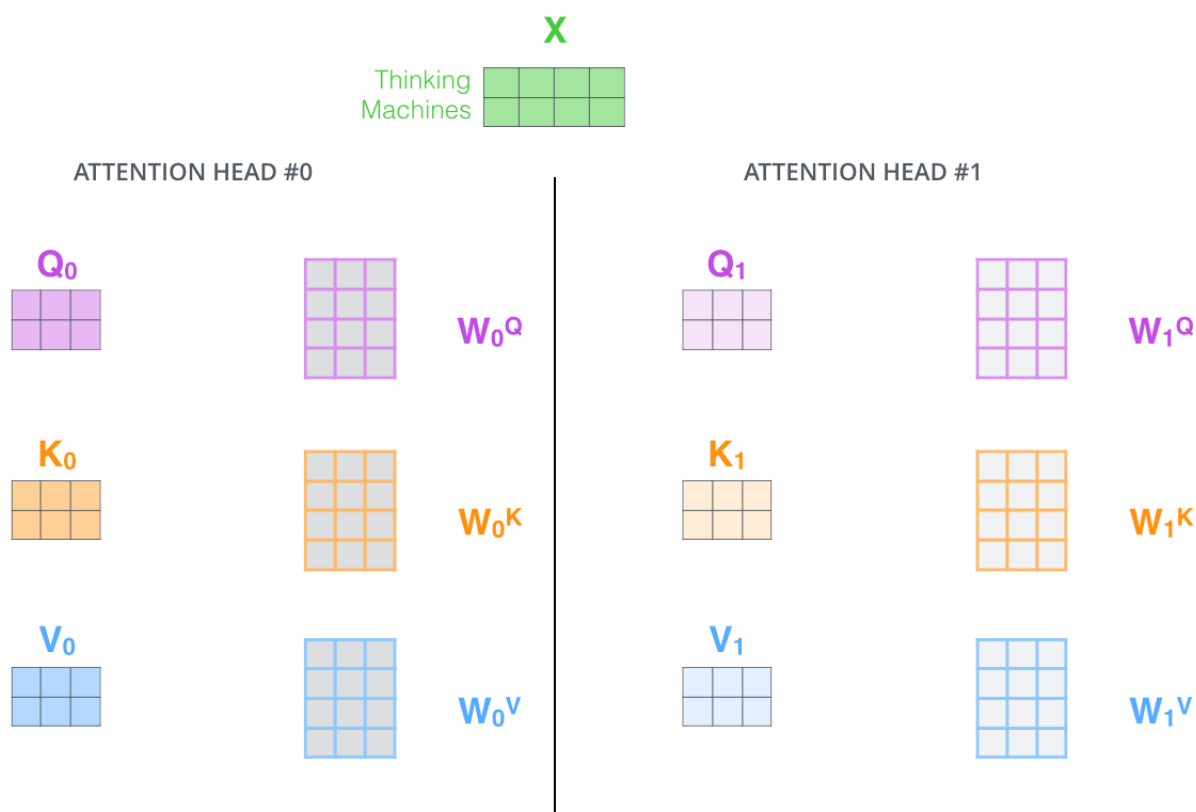


Create a "query", a "key", and a "value" projection of each word (in fact, its embedding vector) in the input sentence, by multiplying with a  $W$  matrix.

# Multi-Head Attention

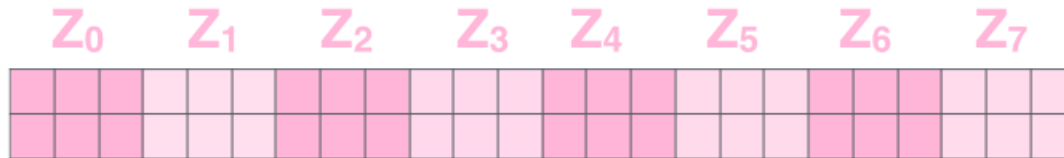
- Instead of performing a single attention pooling, Q-K-V can be transformed with independently learned linear projections.
- Then, these projected Q-K-V are fed into **attention pooling in parallel**.
- In the end, attention-pooling outputs are concatenated and transformed with another learned linear projection to produce the final output.





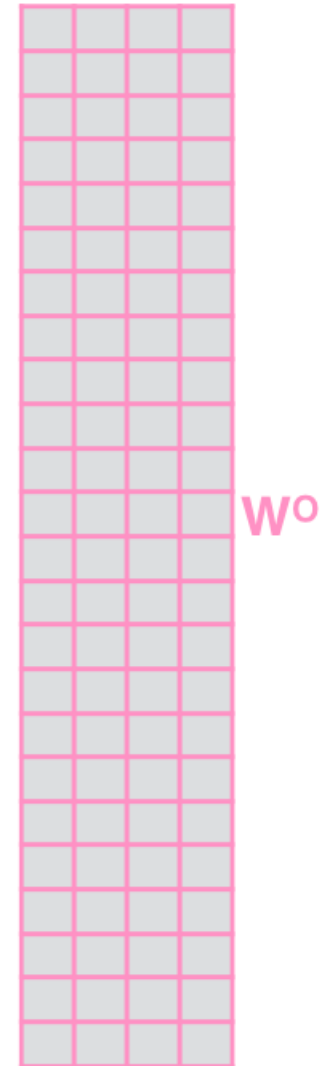
**Example:**  
 If we do the same attention calculation for eight different times with different weight matrices, we end up with eight different  $Z$  matrices. But with multi-head attention, we have different outputs.

1) Concatenate all the attention heads

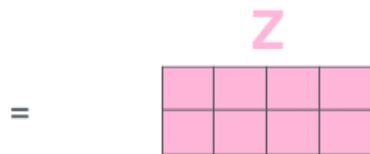


2) Multiply with a weight matrix  $W^O$  that was trained jointly with the model

$\times$



3) The result would be the  $Z$  matrix that captures information from all the attention heads. We can send this forward to the FFNN



To generate the final output, we need to condense all data into a single matrix.

# Transformer: Pipeline of Self-Attention

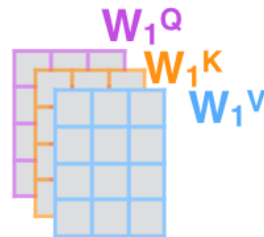
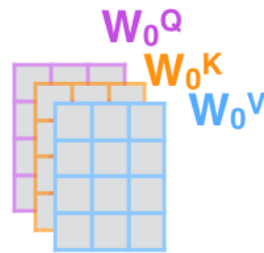
1) This is our input sentence\*

Thinking  
Machines

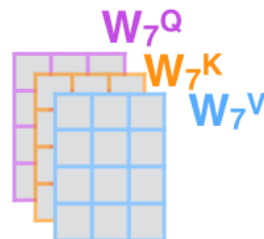
2) We embed each word\*



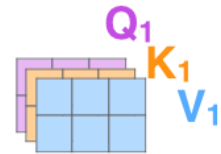
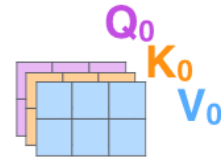
3) Split into 8 heads.  
We multiply  $X$  or  $R$  with weight matrices



...



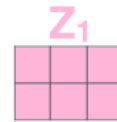
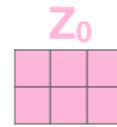
4) Calculate attention using the resulting  $Q/K/V$  matrices



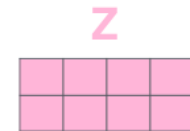
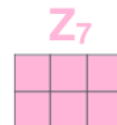
...



5) Concatenate the resulting  $Z$  matrices, then multiply with weight matrix  $W^O$  to produce the output of the layer

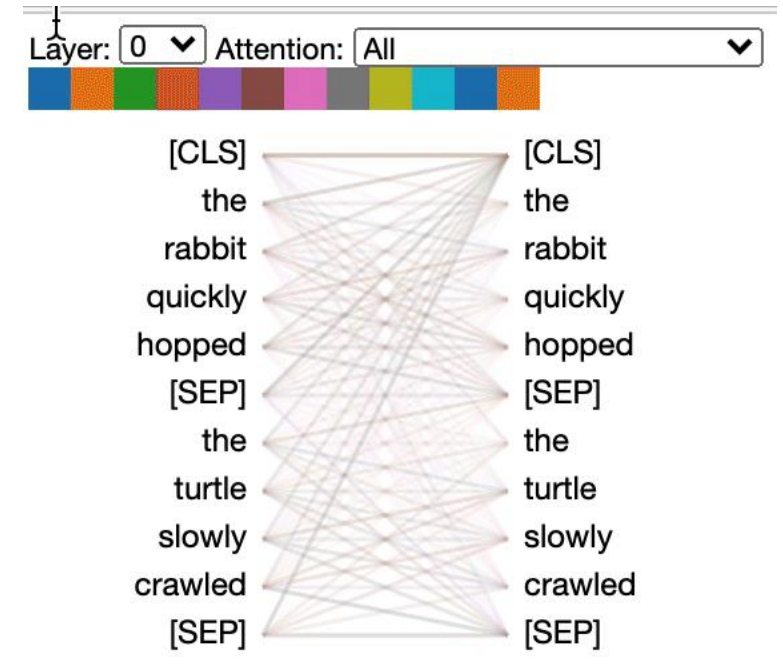


...



# Transformer: Self-Attention

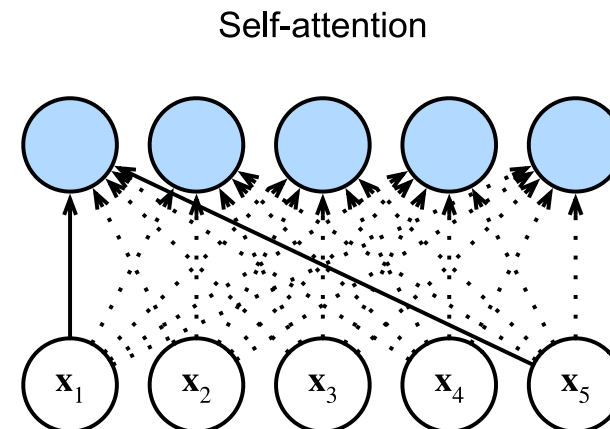
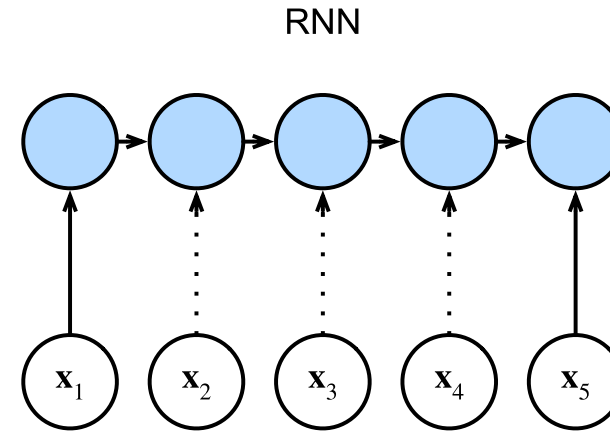
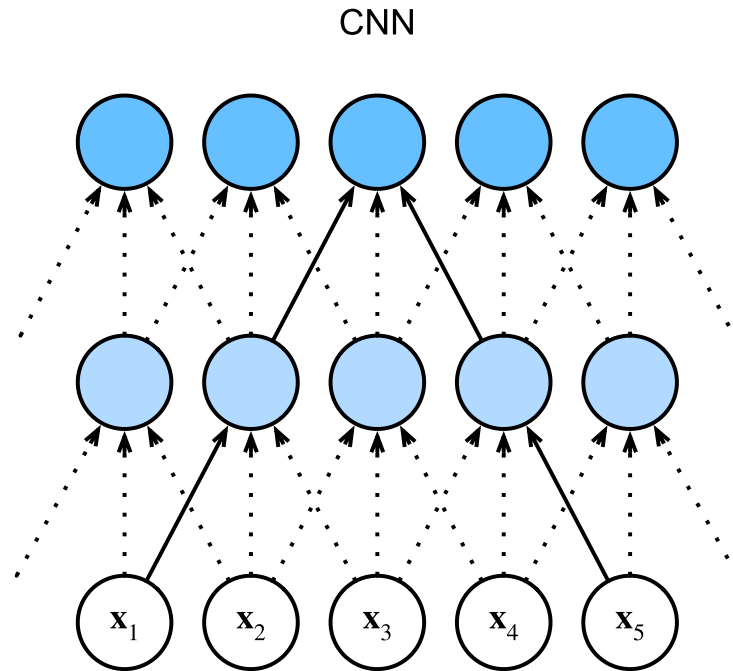
- In Transformer, the term “**self-attention**” refers to the mechanism where each token of an input sequence considers (or “attends to”) all tokens of the sequence, including itself, to compute a new representation.
- The term “self” emphasizes that the model is attending to the input sequence itself, without any external guidance about which parts of the sequence to consider, purely based on learned correlations and dependencies.



<https://github.com/jessevig/bertviz>

# Self-Attention vs. Others

---



# Positional Encoding (PE)

---

- Self-attention by itself does not preserve the order of the sequence.
- **Position encoding**: represents the order of tokens and acts as an additional input associated with each token.
  - The embedded sequence, with position encoding added, is then fed into the encoder part of the Transformer.

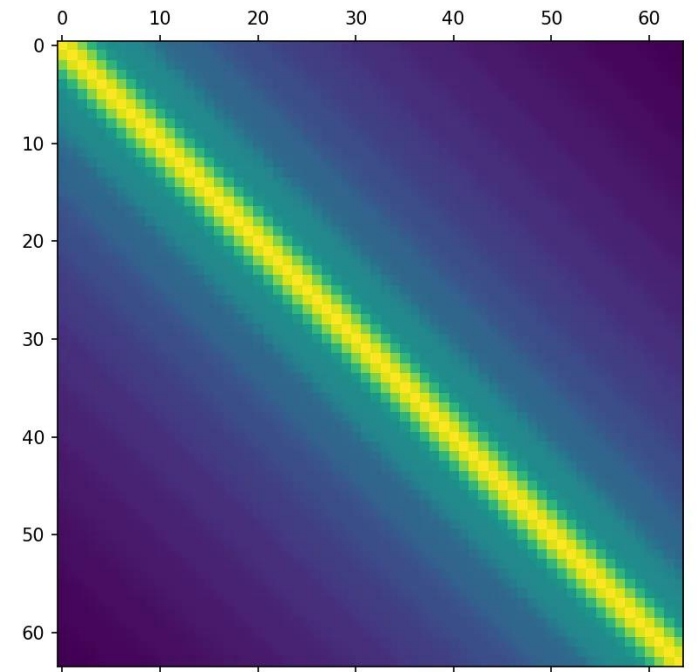
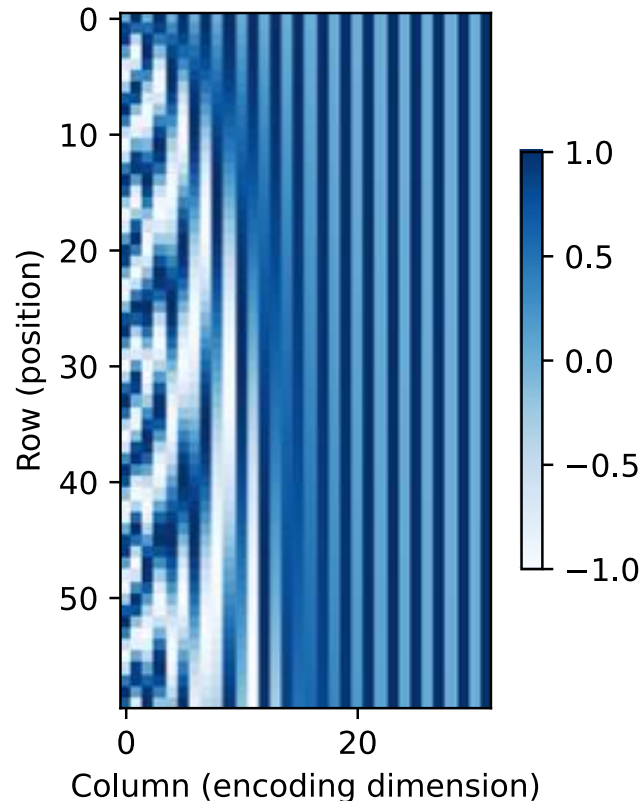
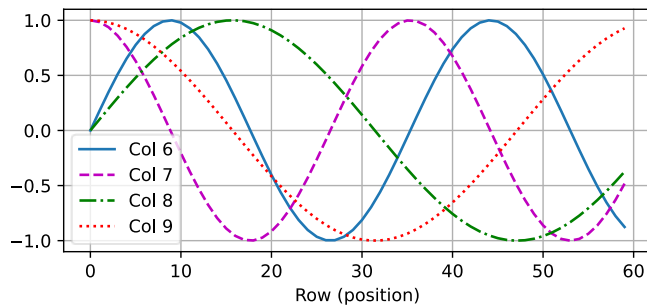


Positional Encoding Matrix for the sequence 'I am a robot'

# Example: Sinusoidal Positional Encoding

- Sinusoidal positional encodings are generated using sine and cosine functions of different frequencies.

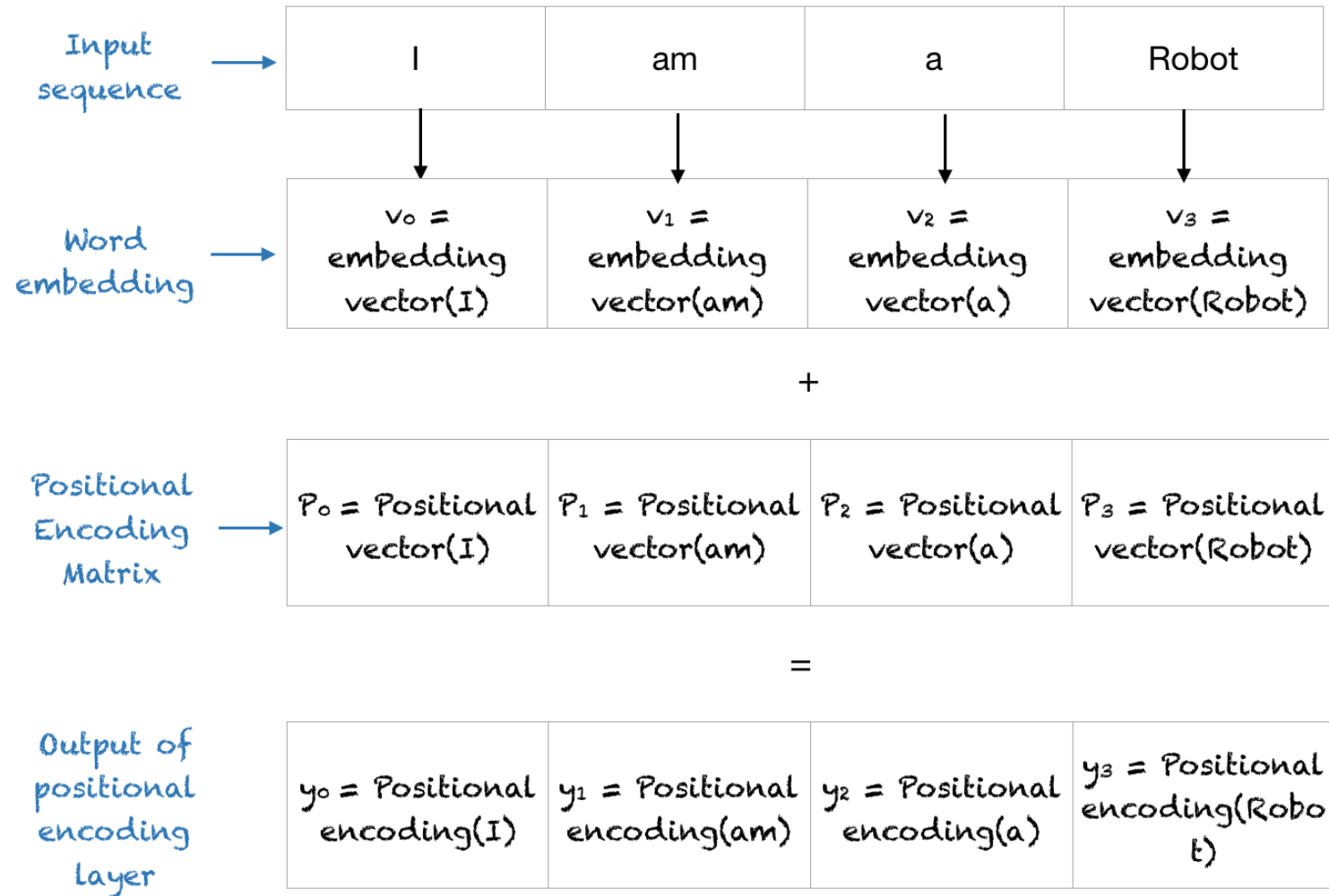
$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right),$$
$$p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right).$$



Relative distance for position encoding of 64-token sequence

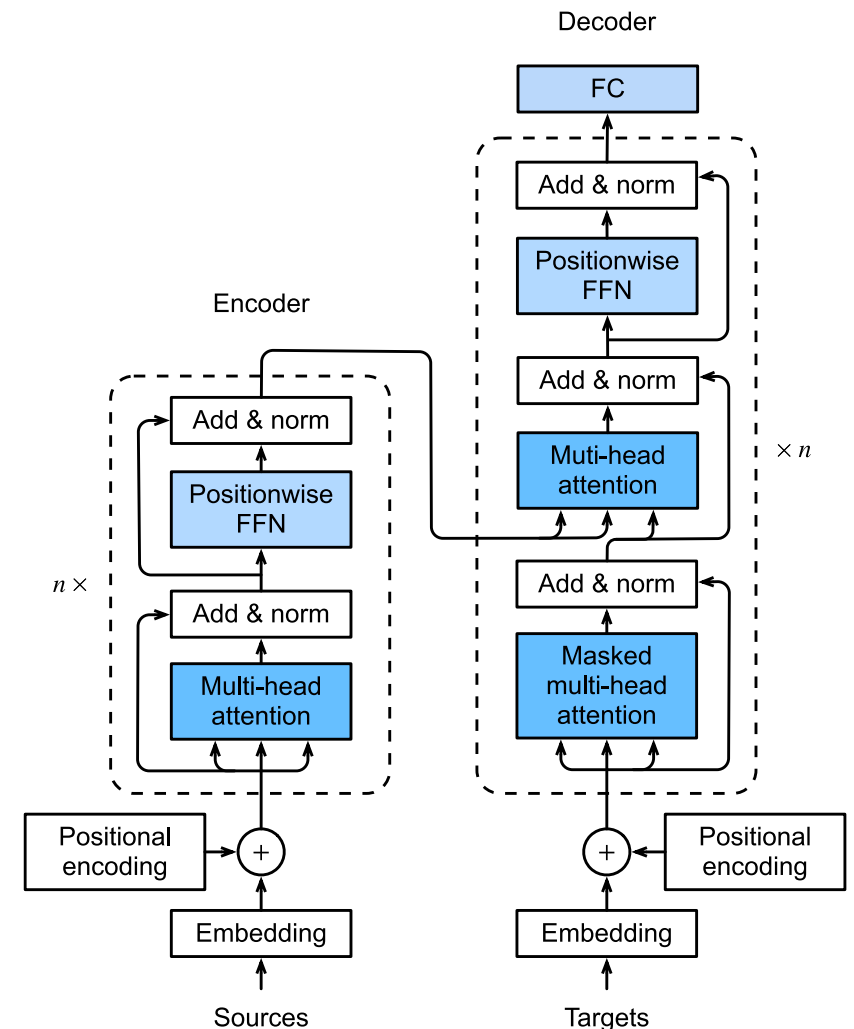
# Example: Sinusoidal Positional Encoding

---



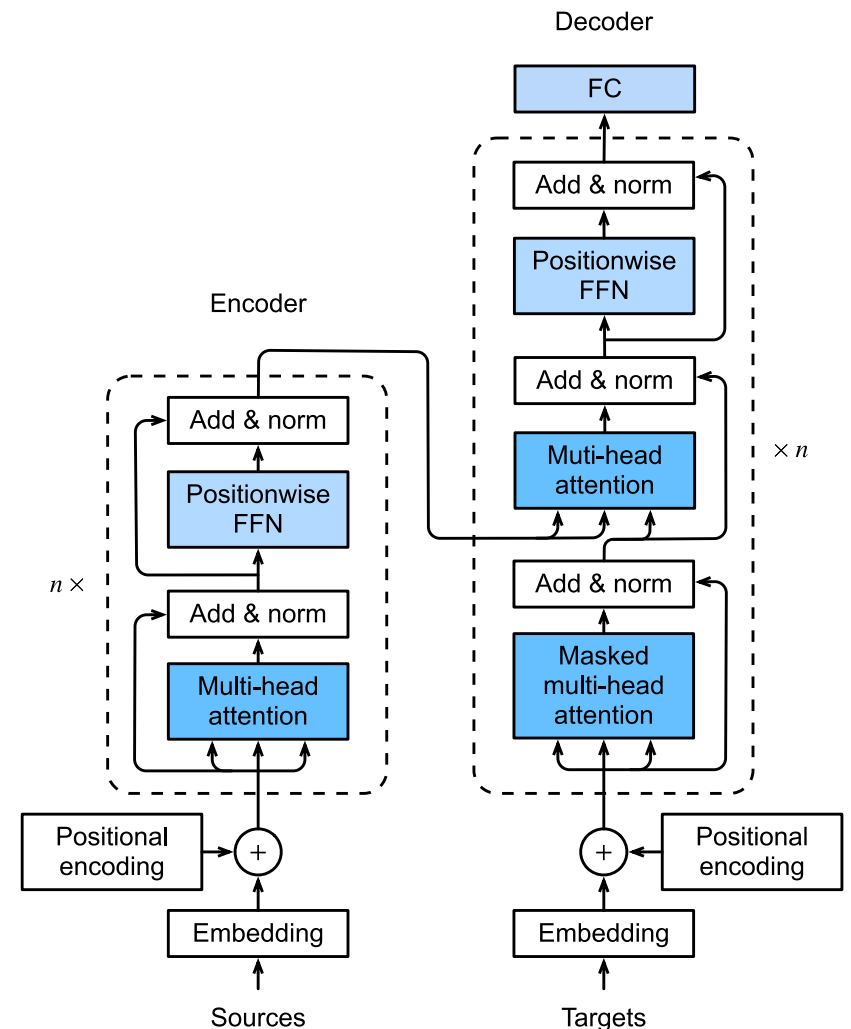
# Transformer: Encoder

- The Transformer encoder is a **stack** of multiple identical layers, where each layer has two sublayers:
  - **Multi-head self-attention**;
  - **Positionwise feed-forward network (FFN)**.
- In the encoder self-attention, Q-K-V are all from the outputs of the previous layer.
- Residual connection is employed.
- Layer normalization (LN) is employed.



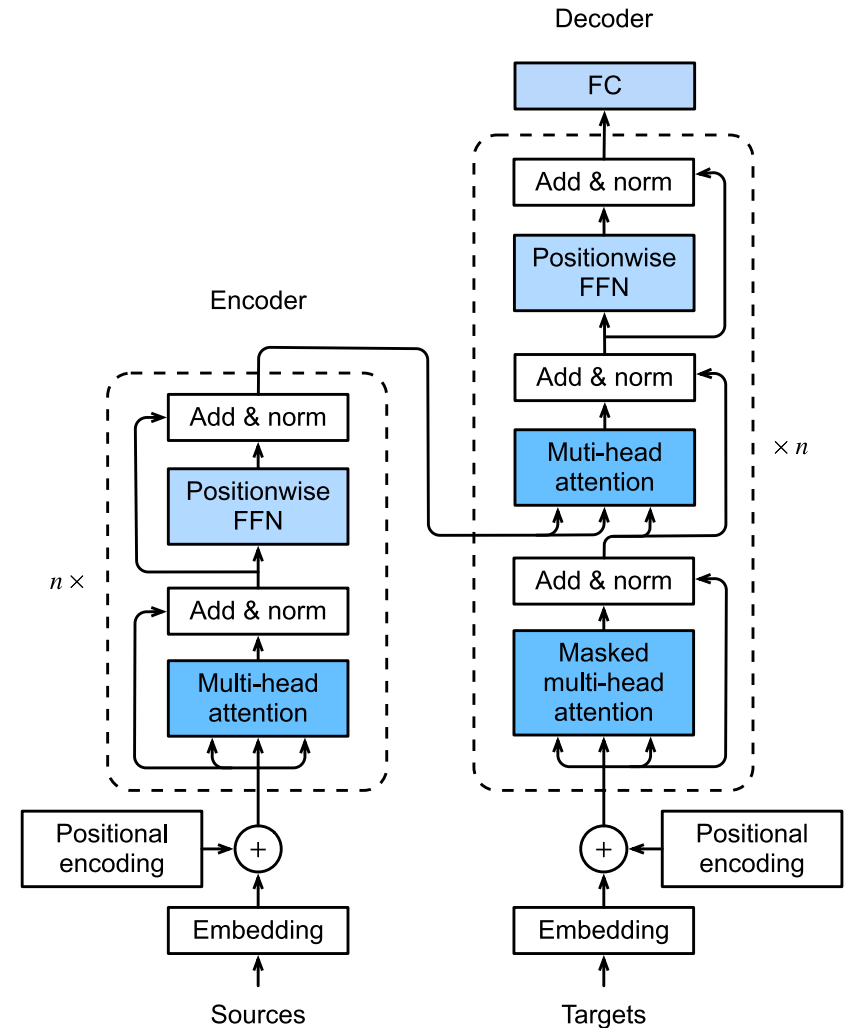
# Transformer: Decoder

- The Transformer decoder is similar to encoder as it consists of multiple identical layers with residual connections and LNs.
- Between the two sublayers as in the encoder, the decoder inserts a third sublayer, known as the **encoder-decoder attention**.
  - Q is from the outputs of the decoder's self-attention sublayer, and K-V are from the Transformer encoder outputs.



# Transformer: Decoder

- In the decoder self-attention, Q-K-V are all from the outputs of the previous decoder layer.
- However, each position in the decoder is allowed only to attend to all positions in the decoder up to that position.
  - This **masked attention** preserves the autoregressive property, ensuring that the prediction can only depend on those output tokens that have already been generated.



# Transformer: Summary

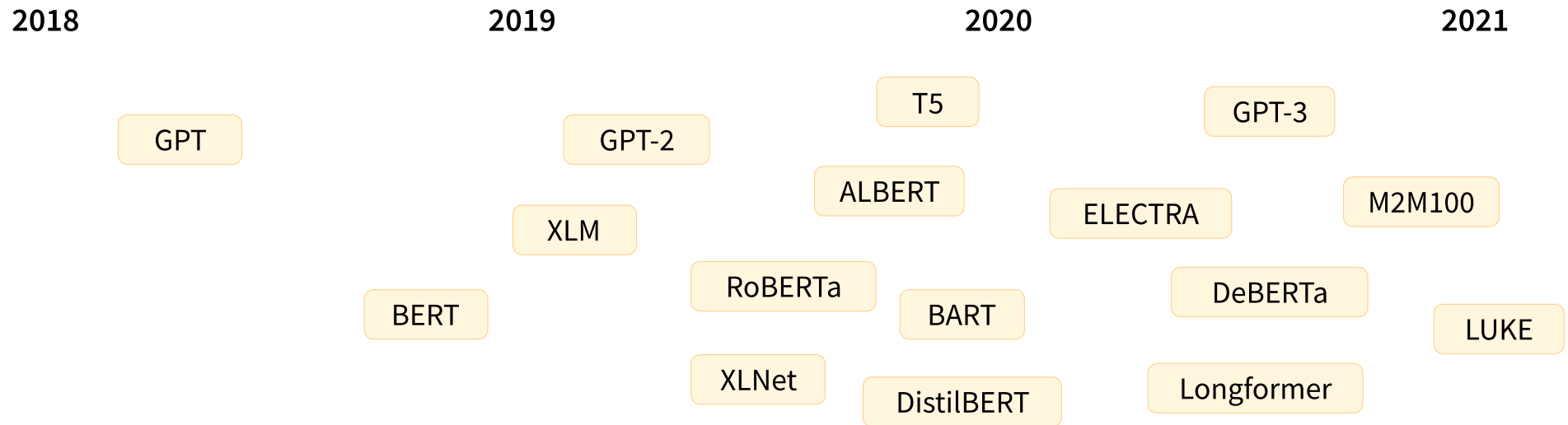
---

- Attention mechanism: The self-attention mechanism allows the model to weigh the importance of different words irrespective of their position, enabling it to capture dependencies without regard to distance.
- Parallelization: Unlike RNNs, Transformers do not rely on sequential processing, allowing parallel computation. This results in faster training and inference times.

# Transformer: Summary

---

- Scalability: They are highly scalable and have been extended to models with billions of parameters, like BERT and GPT.

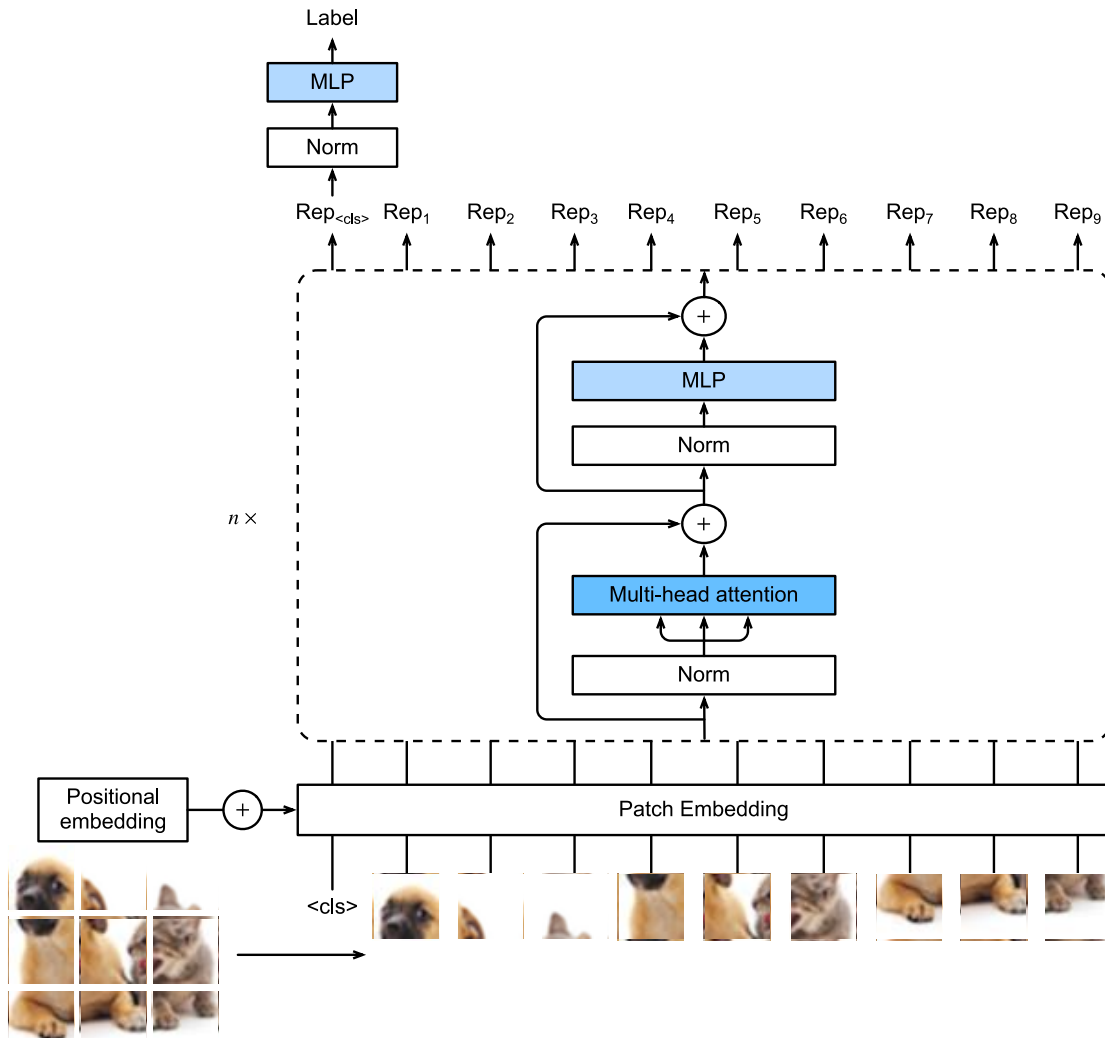


# Vision Transformer (ViT)

---

- The Transformer architecture was initially proposed for NLP tasks.
- In computer vision the dominant architecture has remained the CNN since 2012.
- Is it possible to do better by adapting Transformer models to image data?
  - The answer is ViT, which uses self-attention mechanisms to process images (and other visual) data.

# ViT: Architecture



**Classification:** A special classification token “CLS” is introduced, which attends to all patches and is used to make predictions at the end of the process. The final hidden state corresponding to the “CLS” token is passed through a classification head to predict the image class.

**Self-attention:** The self-attention mechanism allows the model to compute relationships between all image patches. Each patch attends to other patches, allowing the model to capture both local and global features.

**Embedding:** Each patch is linearly embedded into a vector of a fixed size. Note that positional encoding is added.

**Patching:** The input image is split into small, fixed-size patches. These patches are flattened and treated as tokens, similar to words in a sentence.

# ViT: Applications

---

- Image classification: achieves state-of-the-art performance on several benchmarks, particularly when pre-trained on large datasets.
- Object detection: state-of-the-art Detection Transformers (DETR).
- Semantic segmentation: Good, not best yet!
- Image generation: strong capability of modeling complex dependencies to enhance the quality of generated content.
- Medical imaging: widely adopted for various tasks.
- Video understanding: extensions of ViTs have been developed for video analysis tasks, taking into account temporal information along with spatial information from frames.

# Summary

---

- Transformers have become foundational in NLP, leading to breakthroughs in tasks like translation, sentiment analysis, and text generation.
- They have also influenced other fields, particularly computer vision, with adaptations such as Vision Transformers (ViTs).
- The architecture's ability to handle large-scale data and leverage transfer learning has led to the development of prominent foundation models like GPT.

Thank You!